**Minesweeper Final AI Report**

**Team number:** 2
**Member #1 (name/UCI netid):** Eileen Kang (kangeb)
**Member #2 (name/UCI netid):** Nicolas Fuller (fullern1)

## I. Minimal AI

### I.A. Briefly describe your Minimal AI algorithm. What did you do that was fun, clever or creative?

Our Minimal AI algorithm keeps track of the Minesweeper board internally and uses simple logical rules to decide what move to make next. Each tile stores whether it is covered, flagged, already queued for an action, and what number label it has if it has been uncovered.

After every move, the AI updates the board and checks all uncovered numbered tiles. If the number of unknown neighboring tiles matches the remaining mines around that tile, it flags all of those neighbors. If the remaining mine count is zero, it uncovers all of those neighbors because they must be safe.

We used a queue to store future actions so the AI can find multiple safe moves or flags at once and then perform them one at a time. The AI repeatedly applies the rules until no more certain moves can be found. It was almost like it was “thinking through” the board before guessing, which was very interesting to observe. It successfully solved and completed 100% of 1000 easy worlds.

### I.B Describe your Minimal AI algorithm's performance:

| Board Size | Sample Size | Score | Worlds Complete |
|---|---|---|---|
| 5x5 | 1000 | 1000 | 1000 |
| 8x8 | 0 | 0 | 0 |
| 16x16 | 0 | 0 | 0 |
| 16x30 | 0 | 0 | 0 |
| Total Summary | 1000 | 1000 | 1000 |

## II. Final AI

### II.A. Briefly describe your Final AI algorithm, focusing mainly on the changes since Minimal AI:

The Finial AI improves upon the Minimal AI, by “dividing and conquering” with a more complex probability model. Thus, we improve from pure logical deductions to being able to make strong guesses based on the entire board state when there are no clear safe squares or mines.

We use a vector<vector<TileInfo>> to track the state of every tile. When the rules of thumb fail, we use a Breadth-First Search to partition the “frontier” into independent BoardComponent structures. This helps isolate the independent problems, reducing the search space for our solver.

For each component, we begin a recursive search to identify every valid mine configuration. We represent these various configurations using 64-bit bitmasks. To maintain efficiency, the search space is pruned by tracking the remaining mines needed by a number and the available hidden neighbors. Configurations that violate these constraints or exceed the total number of mines left on the board are ignored.

To calculate exact probabilities, we use a dynamic programming approach to merge the independent components and observe how they affect each other. With the combinations calculations (nCr), we figure out how many ways mines can be placed in the remaining empty areas that aren't near any uncovered tiles. The probabilities engine then calculates the probability of every tile being a mine by summing the valid full-board arrangements where a mine exists at that tile, and dividing it by the total possible arrangements of the board.

For guessing, we implement a heuristic that looks one step ahead. If multiple tiles share the minimum mine probability, we calculate the benefit by simulating the result of uncovering each candidate. It generates a distribution for every possible label (0-8) the tile could reveal. For each label, it runs a trial of findSafeMoves to count how many guaranteed actions would follow. The solver then selects the tile that has the maximum expected benefit (the weighted average of future certain moves), prioritizing information gain.

**II.B Describe your Final AI algorithm's performance:**

| Board Size | Sample Size | Score | Worlds Complete |
|---|---|---|---|
| 5x5 | 1000 | 1000 | 1000 |
| 8x8 | 1000 | 905 | 905 |
| 16x16 | 1000 | 871 | 871 |
| 16x30 | 1000 | 469 | 469 |
| Total Summary | 4000 | 3245 | 3245 |

**III. In about 1/4 page of text or less, provide suggestions for improving the performance of your system.**

One big area for improvement is the efficiency of the backtracking solver. Currently, we skip groups bigger than 40 tiles because the search space becomes very large. Implementing a more formal CSP solver or using a more optimized bitmask approach could allow us to solve problems with larger frontiers more precisely. There is also the idea of identifying the bottleneck tiles that split large groups into smaller independent problems, and determining a way to deal with those.

Additionally, when dealing with a tie, our heuristic only looks one move ahead. A more advanced implementation would be to use a Monte Carlo simulation to estimate winning probabilities for various guesses more accurately.